

ECOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA



المدرسة العليا لأساتذة التعليم التقني
المحمدية
جامعة الحسن الثاني بالدار البيضاء

Département Mathématiques & Informatique

BDCC1-CCN1-GLSID1-Master SDIA

Cours Analyse des algorithmes

Cours

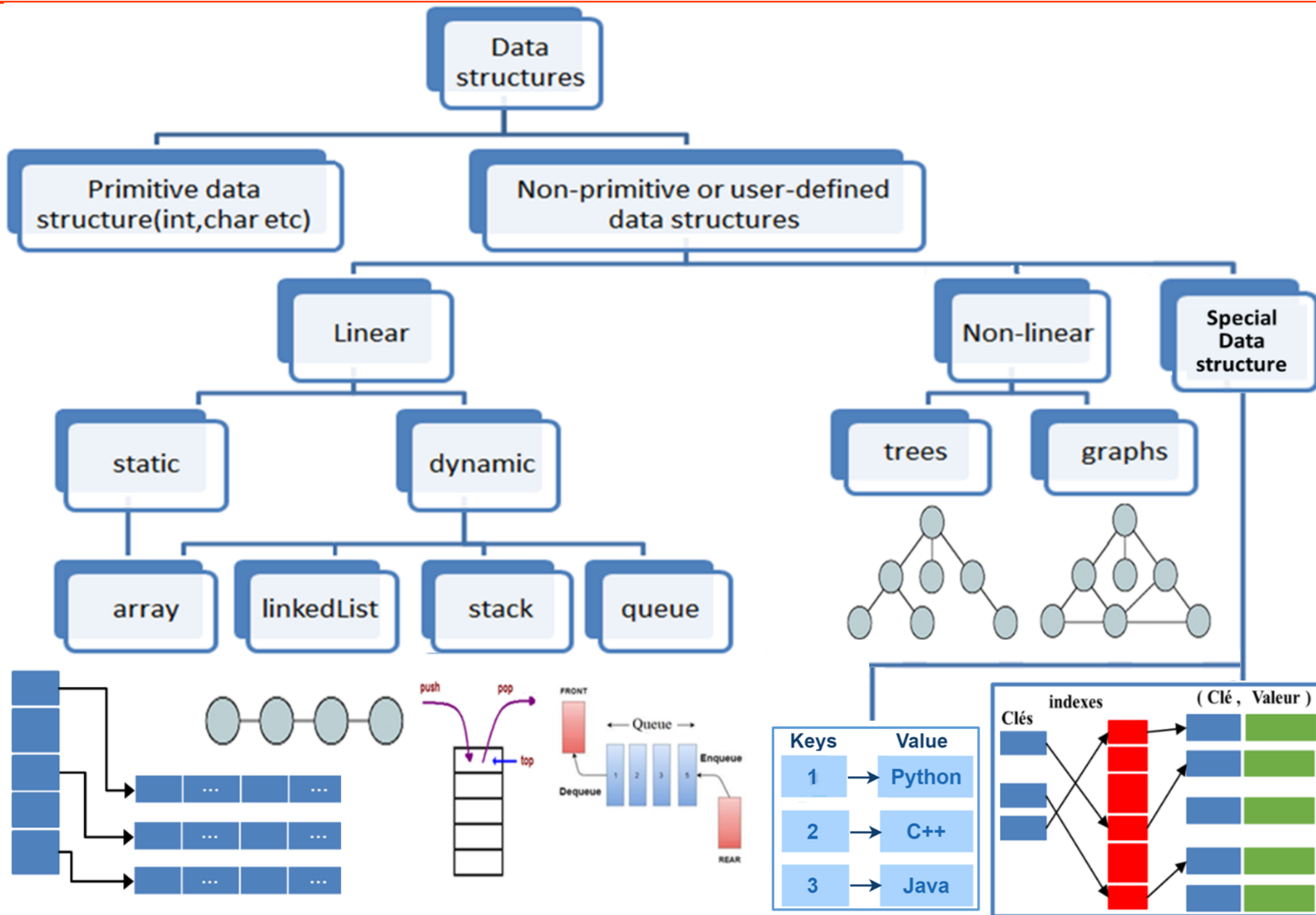
TD

TP

Objectifs :

- ☞ **Analyser l'efficacité d'une structure de données**
 - **Comprendre comment mesurer le coût d'une structure de données ou d'un programme.**
 - **Mesurer les coûts en temps et en mémoire d'une structures de données ou d'un programme,**
 - Calcul théorique et calcul asymptotique
 - Calcul expérimentale
- ☞ **Choisir les structures de données répondre le mieux aux exigences d'une solution efficace d'un problème**
- ☞ **Définir et utiliser des types abstraits de données (TAD)**

Exemples de structures de données



Chapitre II :

Analyse de la complexité des algorithmes

Objectifs :

- ➡ **Etudier la complexité des algorithmes : Mesurer les ressources (temps, mémoire) utilisées par un programme, ou nécessaires à la résolution d'un problème**
 - **Evaluation théorique et calcul asymptotique**
 - **Mesure expérimentale**

Chap II : Analyse de la complexité des algorithmes

- ↗ Introduction - Exemple de problème
- ↗ Méthodes de calcul
 - Méthode expérimentale
 - Méthode théorique
- ↗ Prérequis Mathématiques
- ↗ Complexités représentatives – cas Meilleur, Cas Pire, Moyenne
- ↗ Notation de Landau – Analyse asymptotique
- ↗ Classes de complexité
- ↗ Applications et exercices

I. Introduction

1. Position du problème

Question

Soient A1 et A2 deux algorithmes pour le même problème.

Comment faire le choix de l'une des Solutions A1 et A2?

Réponse

Mesurer le temps $T1(A1)$ et le temps $T2(A2)$ pour les mêmes données et pour différentes configurations et choisir le plus rapide.

=> Définir et utiliser une méthode de mesure pour Mesurer le temps d'un algorithme

Remarques :

On distingue aussi la notion de **complexité spatiale**. Elle correspond à la capacité mémoire exigée par une solution algorithmique.

Cette complexité est moins importante que la **complexité temporelle**. La suite du cours sera consacrée à cette dernière. En effet la complexité temporelle reflète indirectement la complexité spatiale

2. Méthodes de mesure de la complexité Temporelle

Pour évaluer la complexité temporelle d'un algorithme A on distingue deux démarches possibles :

- **Méthode expérimentale (Empirique)**
- **Méthode Théorique (Mathématique)**

II. Méthode expérimentale

1. Principe

Cette méthode consiste à :

- Programmer l'algorithme dans un langage (choix du langage ?)
- Exécuter le programme pour des tailles de données différentes
- évaluer le temps pour chaque exécution.

2. Inconvénients

cette méthode très limitée pour les raisons suivantes :

- Nécessite de programmer l'algorithme
- La mesure dépend de la **machine utilisée**, de **l'environnement d'exécution**, de **l'environnement de programmation** choisi et du **jeu de données** utiliser pour faire la mesure.

3. Démarche

Utiliser les fonctions de relevé du temps système avant et après le programme à mesurer.



4. Outils de travail

Outils C / C++

```
void gettime(struct time *ptemp) // déclarée dans dos.h  
struct time{ unsigned char ti_min,ti_hour,ti_hand,ti_sec};
```

Ou bien

```
time_t time ( time_t * ptrSec );// envoie le temps courant (time.h)  
clock_t clock() //renvoi la durée d'exécution d'un prog (time.h)  
CLK_TCK |CLOCKS_PER_SEC//le nombre de tops d'horloge par seconde
```

Python

- Méthodes `time.perf_counter()` du module `time`

#Méthode `time.perf_counter()`

import time

def print_square(x):

return x ** 2

start = time.perf_counter()

print_square(3)

end = time.perf_counter()

ms = (end-start) * 106**

print(f"Elapsed {ms:.03f} micro secs.")

Python

- Méthode `time_ns()` du module `time`

#Méthode `time_ns()`

`import time`

`def print_square(x):`

`return (x**2)`

`start = time.time_ns()`

`print_square(3)`

`end = time.time_ns()`

`print("Time taken", end-start, "ns")`

Python

- Méthodes `time.time()` du Module `time`

#Méthode `time()`

`import time`

`def print_square(x):`

`return x ** 2`

`start = time.time()`

`print_square(3)`

`end = time.time()`

`ms = (end-start) * 10**6`

`print(f"Elapsed {ms:.03f} micro secs.")`

java

- Fonction `currentTimeMillis` de la Classe `System` :

```
long time1= System. currentTimeMillis();
```

```
    algorithme_appel(...) //appel au programme
```

```
long ecart=(System. currentTimeMillis()-time1)/1000.0;
```

- Classe **Date** appartenant au package `java.util`

```
    Date d= new Date(); long time1=d.getTime();
```

```
    algorithme_appel(...) //appel au programme
```

```
long ecart=(d.getTime()-time1)/1000.0;
```

- Classe **Calendar** appartenant au package `java.util`

```
    Calendar c= Calendar.getInstance();long time1= c.getTimeInMillis();
```

```
    algorithme_appel(...) //appel au programme
```

```
long ecart=c.getTimeInMillis()-time1)/1000.0;
```

5. Exercices

Exercice1

Mesurer le temps nécessaire pour calculer X^N pour les deux schémas de calculs suivants :

- Schéma 1 :

$$P_0 = 1;$$

$$P_i = P_{i-1} * X$$

- Schéma2 :

$$P_0 = 1;$$

$$P_i = P_{i/2} * P_{i/2} \text{ si } i \text{ est pair;}$$

$$P_i = P_{i/2} * P_{i/2} * X \text{ si non}$$

Exercice2

Mesurer le temps nécessaire pour calculer le produit de deux matrices carrées de taille N

Correction de l'exercice1

```
from time import perf_counter
```

```
def puiss_iter(x, n):  
    result = 1  
    for _ in range(n):  
        result *= x  
    return result
```

```
def puiss_rec(x, n):  
    if n == 0:  
        return 1  
    elif n % 2 == 0:  
        temp = puiss_rec(x, n // 2)  
        return temp * temp  
    else:  
        temp = puiss_rec(x, (n - 1) // 2)  
        return x * temp * temp
```

```
temps_debut_iter = perf_counter()  
puiss_iter(1, 1000000000000)  
temps_fin_iter = perf_counter()  
temps_iter = temps_fin_iter - temps_debut_iter
```

```
temps_debut_rec = perf_counter()  
puiss_rec(1, 1000000000000)  
temps_fin_rec = perf_counter()  
temps_rec = temps_fin_rec - temps_debut_rec  
  
print(f"Temps puiss_iter:{temps_iter} sec")  
print(f"Temps puiss_rec:{temps_rec} sec")
```

Correction de l'exercice1

```
import java.util.Calendar;
public class MesurePuissance{
    public static double puis2(double x,long n){
        if(n==0)return 1;
        double pdemi=puis2(x,n/2);
        if(n%2==0)return pdemi*pdemi;
        return pdemi*pdemi*x;
    }

    public static double puis1(double x,long n){
        double p=1; long i;
        for(i=1;i<=n;i++) p*=x;
        return p;
    }

    public static String toString(double d){
        final long nbSecJour=24*3600L;
        String s =new String();
        if(d>=nbSecJour){
            s=(long)d/nbSecJour + "Jours ";
            d=d-((long)d/nbSecJour)*nbSecJour;
        }
        if(d>=3600){
            s=s+(long)d/3600 +" Heures ";
            d=d-((long)d/3600)*3600;
        }
    }
}
```

```
        if(d>=60){
            s=s+(long)d/60 +" Minutes ";
            d=d-((long)d/60)*60;
        }
        d=(long)(d*1000);d=d/1000; s=s+d+"Sec";
        return s;
    }

    public static void main(String[] args){
        double x=1.0; long n=100000000000L;
        Calendar c=Calendar.getInstance();
        long t1=c.getTimeInMillis();
        puis1(x,n);
        long t2=c.getTimeInMillis();
        System.out.println("le temp puissance pour
x="+x +" et n="+n +" : ");
        System.out.println("puis1 : "
+toString((t2-t1)/1000.0));
        t1=c.getTimeInMillis();
        puis2(x,n);
        t2=c.getTimeInMillis();
        System.out.println("puis2 : "
+toString((t2-t1)/1000.0));
    }
}
```


Temps obtenus pour X^N pour $N=1000000000000.0$

```
le temps de calcul de la puissance pour x=1.0 et n=1000000000000 :  
puis1 : 7 Minutes 31.994 Secondes  
puis2 : 0.0 Secondes
```

Correction de l'exercice2 – Solution 1 basée sur les tableaux (arrays) et la méthode dot fournis par numpy

```
import numpy as np  
from time import perf_counter  
  
# Générer des matrices aléatoires  
taille_A = (3000, 100) # taille de la matrice A  
taille_B = (100, 1000) #taille de la matrice B  
A = np.random.rand(*taille_A)  
B = np.random.rand(*taille_B)  
  
temps_debut = perf_counter()  
np.dot(A, B)  
temps_fin = perf_counter()  
temps = tempsFin - tempsDebut  
  
print(f"Temps du produit matriciel : {temps} sec")
```

Correction de l'exercice2 – Solution 2 basée sur les listes python

```
from time import perf_counter
import random

def produit_matriciel(A, B):
    lgs_A, cls_A = len(A), len(A[0])
    lgs_B, cls_B = len(B), len(B[0])
    assert cls_A == lgs_B, "matrices non conformes pour le produit matr."
    resultat = [[0 for _ in range(cls_B)] for _ in range(lgs_A)]
    for i in range(lignes_A):
        for j in range(colonnes_B):
            for k in range(colonnes_A):
                resultat[i][j] += A[i][k] * B[k][j]
    return resultat

dim_A = (3000, 100) # taille de la matrice A
dim_B = (100, 1000) # taille de la matrice B
A = [[[random.random() for _ in range(dim_A[1])] for i in range(dim_A[0])]
B = [[[random.random() for j in range(dim_B[1])] for i in range(dim_B[0])]

temps_debut = perf_counter()
produit_matriciel(A, B)
temps_fin = perf_counter()
print(f"Temps d'exécution : {temps_fin- temps_debut} secondes")
```

Correction de l'exercice2

```
import java.util.Random;
class ProdMat {
static void initMat(float[][] m){
    Random rnd=new Random();
    for(int i=0;i<m.length;i++)
        for(int j=0;j<m[0].length;j++)
            m[i][j]=rnd.nextFloat()*100;
}

static String toString(float[][] m){
    String s="";
    for(int i=0;i<m.length;i++){
        s+=""+m[i][0];
        for(int j=1;j<m[0].length;j++)
            s+=" ,  "+m[i][j];
        s+="\n";
    }
    return s;
}
```

```
static float[][] prodMat(float[][]
    m1,float[][] m2)throws Exception{
    if(m1[0].length!=m2.length) throw new
    Exception("produit mat. impossible");
    float[][] r=new
        float[m1.length][m2[0].length];
    for(int i=0;i<m1.length;i++)
        for(int j =0;j<m2[0].length;j++){
            r[i][j]=0;
            for(int k=0;k<m1[0].length;k++)
                r[i][j]+=m1[i][k]*m2[k][j];
        }
    return r;
}
```

```
public static void main(String[] args){
    float[][] mat1=new float[3000][100],mat2=new float[100][1000];
    initMat(mat1); initMat(mat2);
    try{
        long t1=System.currentTimeMillis();
        prodMat(mat1, mat2);
        long t2=System.currentTimeMillis();
        System.out.println("le temps de calcul du produit matriciel
                           [3000][100]*[100][1000]: ");

        System.out.println("\n\t" +((t2-t1)/1000.0));
    }catch(Exception ex){    ex.printStackTrace(); }
}
```

Temps obtenus

Pour le produit d'une matrice [3000][50] par une matrice [50][1000]

```
le temps de calcul du produit Matricie :
0.165 Secondes
BUILD SUCCESSFUL (total time: 25 seconds)
```

6. Application

Ecrire une application programmée en C/C++ pour comparer pour des tailles différentes les méthodes de tri suivantes :

- Tri bulle
- Tri par insertion
- Tri par sélection
- Tri par fusion
- Tri rapide
- (Tri par tas)

L'application doit représenter les résultats des mesures sous forme graphique.

III. Méthode théorique

1. Prérequis mathématiques

Propriétés des fonctions Logarithmes :

Pour toutes valeurs positives de m, n et r, et tous entiers positifs a et b:

Binary logarithm $\lg n = \log_2 n$

Natural logarithm $\ln n = \log_e n$

$$\lg^k n = (\lg n)^k$$

$$\lg \lg n = \lg(\lg n)$$

$$\log x^y = y \log x$$

$$\log xy = \log x + \log y$$

$$\log \frac{x}{y} = \log x - \log y$$

$$a^{\log_b x} = x^{\log_b a}$$

$$\log_b x = \frac{\log_a x}{\log_a b}$$

Propriétés des exposants:

$$a^b * a^c = a^{(b+c)}$$

$$(a^b)^c = a^{bc}$$

$$a^b / a^c = a^{(b-c)}$$

$$b = a^{(\log_a b)}$$

$$b^c = a^{(c * \log_a b)}$$

III. Méthode théorique

Sommations usuelles :

- Arithmetic series:

$$\sum_{k=1}^n k = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

- Geometric series:

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \dots + x^n = \frac{x^{n+1} - 1}{x - 1} \quad (x \neq 1)$$

- Special case: $|x| < 1$:

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x}$$

- Harmonic series:

$$\sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \dots + \frac{1}{n} \approx \ln n$$

- Other important formulas:

$$\sum_{k=1}^n \lg k \approx n \lg n$$

$$\sum_{k=1}^n k^p = 1^p + 2^p + \dots + n^p \approx \frac{1}{p+1} n^{p+1}$$

$$\sum_{i=1}^n \frac{1}{2^i} = 1 - \frac{1}{2^n},$$

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1.$$

$$\sum_{i=1}^n \frac{i}{2^i} = 2 - \frac{n+2}{2^n}.$$

2. Principe

Consiste à évaluer la complexité d'un algorithme par le **décompte** des **instructions de base** de l'algorithme et non pas par sa mesure en unité de temps (par exemple les secondes).

Cette méthode consiste à :

- Déterminer une grandeur entière **N** qui quantifie la **taille des données de l'algorithme** (taille d'un tableau, degré polynomial, Nbre d'éléments dans une matrice, Nbre de bits dans la représentation binaire de l'entrée, Nbre de sommets et d'arêtes dans un graphe, ...)
- Déterminer les **opérations fondamentales** qui constituent les **opérations déterminantes (à compter)** pour l'algorithme,
- Faire le **décompte** des opérations fondamentales en fonction de **N** en suivant des **règles de décomptage**.

Le résultat constitue alors la **complexité théorique notée $T(N)$** .

Exemples

- déterminer une grandeur entière **N** qui quantifie la **taille des données de l'algorithme**

Algorithme	Taille N
------------	----------

- Déterminer les **opérations fondamentales** qui constituent des **opérations déterminantes** pour l'algorithme

Algorithme	Oper. Fondamentale(s)
------------	-----------------------

3. Règles de calcul de T(N)

Pour déterminer T(N) on s'appuie sur les règles suivantes :

- **Rg1 : Opération élémentaire**

$$T(\text{opération fondamentale}) = \text{constant}$$

- **Rg2 : Séquence d'opérations**

$$T(seq) = \sum_{oper \in Seq} T(oper)$$

la complexité d'une suite est la somme des complexités de ses actions.

- **Rg3 : conditionnelle et sélective**

$$T(\text{si } C \text{ Alors } A \text{ Sinon } B) = T(C) + \text{Max}(T(A), T(B))$$

$$\begin{aligned} T(\text{switch}(S)\{\text{case } V_1: A_1; \text{case } V_2: A_2; \dots, \text{case } V_n: A_n; \text{default: } A_{n+1}\}) \\ = T(S) + \sum_{i=1..n} T(V_i) + \text{Max}_{i=1..n}(T(A_i)) \end{aligned}$$

- **Rg4 : Boucle for(E1,E2,E3)B;**

Soit $T_i(B)$ la complexité du bloc B de la boucle pour le $i^{\text{ième}}$ passage, la complexité de la boucle est donnée par :

$$T_{\text{boucle}} = T(E1) + \sum_{i=1}^n (T_i(B) + T(E2) + T(E3)) + T(E2)$$

Où n est le nombre de passages de la boucle

- **Rg5 : Fonction/procédure**

leur complexité est déterminée par **celle de leur corps**.

Dans le cas de la **rékursivité**, le temps de calcul est exprimé comme une **relation de récurrence (à trouver et à résoudre)** de la forme :

$$\begin{cases} T(n) = f(T(k)), k < n \text{ et } n > n_0 \\ T(n) = C_{n_0}, n \leq n_0 \end{cases}$$

Où n et k sont les tailles pour les appels rékursifs successifs. $N \leq n_0$ étant les tailles pour l'arrêt des appels rékursifs.

Exemple : Calcul de la Puissance

Solution 1 :

```
double puis1(double x, long n){  
    double p=1; long i;  
    for(i=1;i<=n;i++)  
        p*=x;  
    return p;  
}
```

Solution 2 :

```
double puis2(double x, long n){  
    if(n==0)return 1;  
    double pdemi=puis2(x,n/2);  
    if(n%2==0)return pdemi*pdemi;  
    return pdemi*pdemi*x;  
}
```

- Taille du problème → Exposant n,
- Opération fondamentale → *

$$T_1(n) = \sum_{i=1}^n 1 = n$$

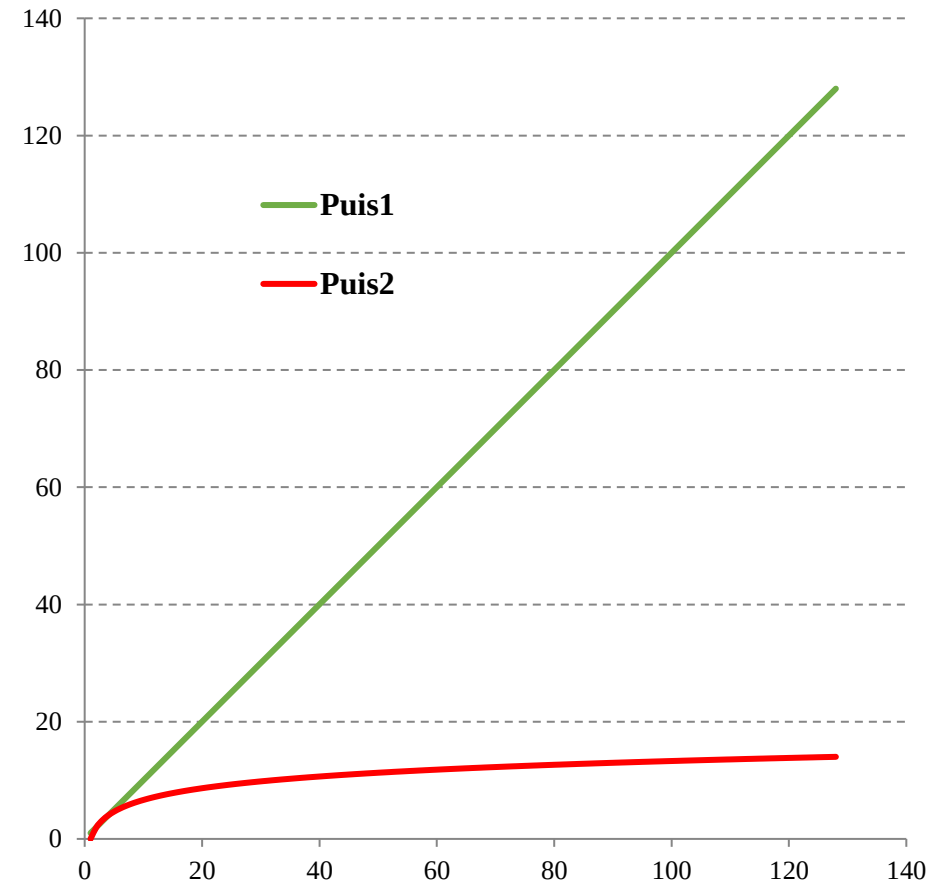
$$\begin{cases} T_2(n) = T_2(n/2) + 2 \quad \forall n > 0 \\ T_2(0) = 0 \end{cases}$$

- **Résolution de l'équation de récurrence :**

$$\begin{cases} T_2(n) = T_2(n/2^1) + 2 \\ T_2(n/2^1) = T_2(n/2^2) + 2 \\ \dots \\ T_2(n/2^p) = T_2(0) + 2 \\ T_2(0) = 0 \end{cases}$$

Où p est tel que $2^p = n \rightarrow p = \log_2 n$

$$T_2(n) = \sum_{i=0}^p 2 = 2(p+1) = 2 + 2\log_2 n$$



Exercices

Exercice 1

1. Rappeler l'algorithme relatif au jeu de Hanoï qui consiste à déplacer N disques placés dans une tige par ordre décroissant du rayon du bas en haut, vers une 2^{ième} tige en ayant la possibilité d'utiliser une seule tige intermédiaire. On ne pourra déplacer qu'un seul disque à la fois. On n'a pas le droit de poser un disque plus grand sur un disque plus petit. On suppose que l'opération de déplacement d'un disque est primitive et est déjà définie.
2. Evaluer sa complexité

Exercice 2

1. Donner les solutions itérative et récursive qui calculent le nième terme de la suite de Fibonacci définie par :

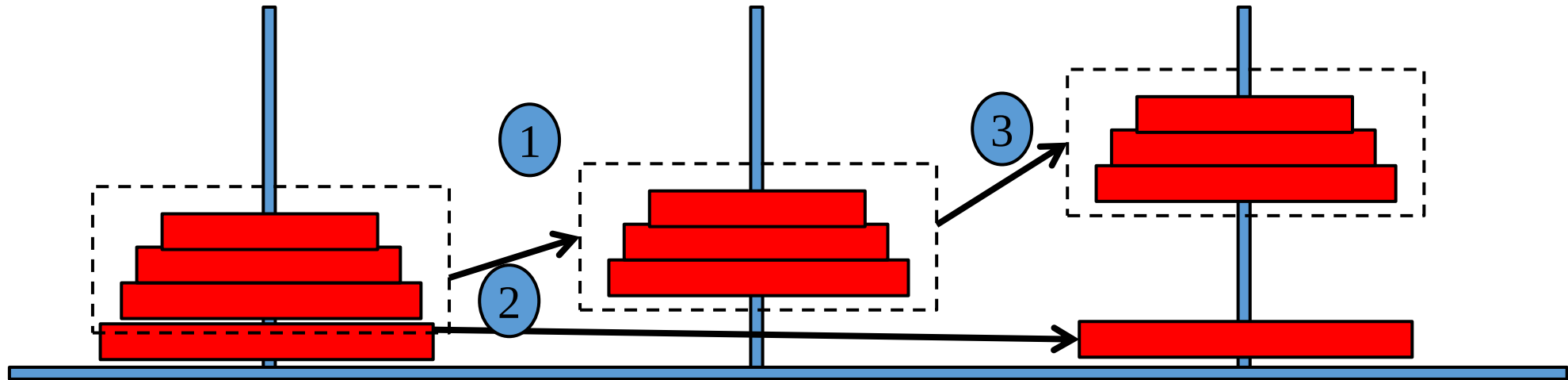
$$U_0=0, U_1=1, U_n=U_{n-2}+U_{n-1} \quad \forall n>1$$

1. Evaluer leurs complexités.

Correction :

Exercice 1

Principe :



Solution

```
void Deplacer(int NbreDis, int NumTigSrc, int NumTigDest){  
    if(NbreDis>0) {  
        Deplacer(NbreDis-1,NumTigSrc,6-NumTigSrc-NumTigDest);  
        DeplacerDisque(NumTigSrc,NumTigDest);  
        Deplacer(NbreDis-1,6-NumTigSrc-NumTigDest,NumTigDest);  
    }  
}
```

Chap I : Analyse de la complexité des algorithmes

- Taille du problème → Nombre de disques NbreDis noté N,
- Opération fondamentale → DeplacerDisque(Tsrc,Tcibl)
- Le traitement est récursif → Trouver une équation de récurrence :
- Soit $T(N)$ la complexité du traitement Deplacer(int N, ...),
 - $T(N-1)$ est alors la complexité pour Deplacer(int N-1,...)
 - d'où la relation :
$$\begin{cases} T(n) = 2 * T(n-1) + 1 & \forall n > 1 \\ T(1) = 1 \end{cases}$$
- **Résolution de l'équation de récurrence :**

$$\begin{cases} T(n) = 2 * T(n-1) + 1 \\ T(n-1) = 2 * T(n-2) + 1 \\ \dots \\ T(2) = 2 * T(1) + 1 \\ T(1) = 1 \end{cases}$$

→ Par substitution on obtient :

$$\begin{aligned} T(n) &= 2(2 * T(n-2) + 1) + 1 \\ &= 2^2 * T(n-2) + 2^1 + 2^0 \\ &= 2^2 * (2 * T(n-3) + 1) + 2^1 + 2^0 \\ &= 2^3 * T(n-3) + 2^2 + 2^1 + 2^0 \\ &= 2^{n-1} * T(1) + 2^{n-2} + \dots + 2^1 + 2^0 \\ &= 2^{n-1} + 2^{n-2} + \dots + 2^1 + 2^0 \end{aligned}$$



$$T(n) = 2^n - 1$$

4. Complexités représentatives

$T(n)$ ne peut être déterminée parfaitement. En effet, pour le même algorithme, elle peut **changer en fonction de la configuration des données entrées**. Par conséquent, il est alors nécessaire de distinguer les **trois situations représentatives suivantes** :

- Le coût dans le **meilleur cas** noté $T_{\min}(N)$: **limite inférieure** de $T(n)$
- Le coût dans le **pire cas** noté $T_{\max}(N)$: **limite supérieure** de $T(n)$
- Le coût dans le **cas moyen** noté $T_{\text{moy}}(N)$: **prédiction** de $T(n)$ par un calcul statistique pour une entrée aléatoire

On doit vérifier que :

$$T_{\min}(N) \leq T(N) \leq T_{\max}(N) \qquad T_{\min}(N) \leq T_{\text{moy}}(N) \leq T_{\max}(N)$$

Exemple : Recherche séquentielle dans un tableau $\rightarrow$ Deux cas limites :

- Valeur recherchée au début du tableau $\rightarrow T_{\min}(N)=1$
- Valeur recherchée n'est pas dans le tableau $\rightarrow T_{\max}(N)=N$

4.1 Définitions

Soient D_N et $T_A(d, N)$ respectivement l'ensemble des entrées de taille N et la complexité temporelle de l'algorithme A pour une entrée $d \in D_N$.

Complexité dans le meilleur cas

Notée $T_{\min}(N)$ est définie par : $T_{\min}(N) = \text{Min}\{T_A(d, N), d \in D_N\}$

Complexité dans le pire cas

Notée $T_{\max}(N)$ est définie par : $T_{\max}(N) = \text{Max}\{T_A(d, N), d \in D_N\}$

Complexité dans cas moyen

Notée $T_{\text{moy}}(N)$ est définie par : $T_{\text{moy}}(N) = \sum_{d \in D_N} p(d) T_A(d, N)$

où $p(d)$ est la **probabilité associée** à d

Si toutes les configurations sont **équiprobables**, la complexité cas

moyen est simplifiée par :

$$T_{\text{moy}}(N) = \frac{\sum_{d \in D_N} T_A(d, N)}{|D_N|}$$

$|D_N|$: Cardinal de D_N = Nombre de configurations possibles

Remarques

- $T_{\min}(N) \leq T_{\text{moy}}(N) \leq T_{\max}(N)$
- Si l'algorithme ne dépend que de la taille N , les trois complexités représentatives sont confondues.

4.2. Exemples

- Recherche séquentielle

```
public static int rechSeq(int[] T, int x){
    int i ;
    for(i=0;i<T.length;i++){
        if(T[i] == x) return i;
    }
    return -1;
}
```

Meilleur cas : x au début de T

$$T_{\min}(N) = 1$$

Pire cas : x n'appartient pas à T

$$T_{\max}(N) = N$$

- Recherche dichotomique

```
public static int rechDich(int[] T, int x){
    int i , g=0,d=T.length-1,m;
    while(g<=d){
        m=(g+d)/2;
        if(T[m] == x) return m;
        if(T[m]>x) d=m-1;
        g=m+1;}
    return -1;
}
```

$$T_{\min}(N) = 1$$

$$T_{\max}(N) = \log_2 N$$

Cas moyen : Exemple de calcul

- On se place dans les deux cas limites suivants : $x \in T$ et $x \notin T$.
- soit q la **probabilité** associée au cas $x \notin T$, la probabilité associée au cas $x \in T$ est alors donnée par $1-q$.
- le cas $x \in T$ peut être divisé en N cas suivants : $x = T[i]$, $i = 0 \dots N-1$
- Si on considère que ces N cas sont équiprobables càd que la probabilité associée à un cas est : $(1-q)/N$. On peut alors écrire :

$$\begin{aligned}
 T_{moy}(N) &= qN + \frac{1-q}{N} \sum_{i=0}^{N-1} (i+1) & T_{moy}(N) &= q \log_2 N + \frac{1-q}{\log_2 N} \sum_{i=1}^{\log_2 N} i \\
 &= qN + \frac{1-q}{N} \sum_{i=1}^N i & &= q \log_2 N + \frac{1-q}{\log_2 N} \frac{\log_2 N (1 + \log_2 N)}{2} \\
 &= qN + \frac{1-q}{N} \frac{N(N+1)}{2} & &= q \log_2 N + \frac{(1-q)(1 + \log_2 N)}{2} \\
 &= qN + \frac{(1-q)(N+1)}{2}
 \end{aligned}$$

IV. Complexité asymptotique

1. Principe

Le calcul de $T(N)$ peut être **simplifié** par une fonction $f(N)$ qui a le **même comportement** que $T(N)$ quand N **augmente infiniment** ($n \rightarrow \infty$). La fonction $f(N)$ est appelée **Complexité asymptotique**

2. Notation grand-O

Soit $T(n)$ une fonction non négative. On dit que $T(N)$ est un $O(f(N))$ s'il existe deux constantes positives C et N_0 telle que :

$$T(N) \leq C f(N) \text{ pour tout } N > N_0.$$

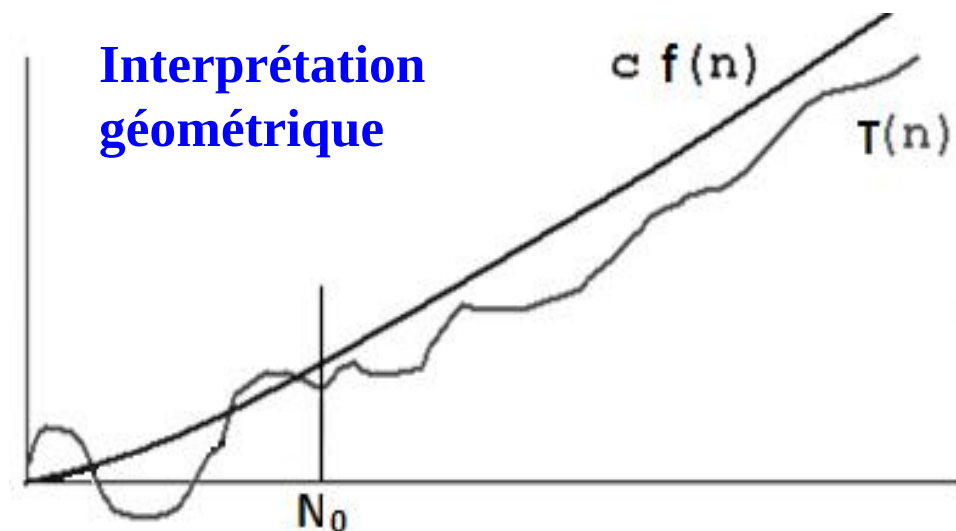
On dit que $T(N)$ est **dominée asymptotiquement** par $f(N)$: ($T(N) \leq f(N)$)

On a :

$$O(1) < O(\log n) < O(n)$$

$$O(n) < O(n \log n) < O(n^2)$$

$$O(n^2) < O(n^3) < O(2^n) \dots$$



Exemples

- Constantes :

$$10 = O(1) , \quad 1027 = O(1)$$

- Si $T(N) = 3N^2 + 2$ alors $T(N) = O(N^2)$.

- Algorithme** **Coût**

som = 0; --> C_1

for(i=0; i<N; i++) --> C_2

for(j=0; j<N; j++) --> C_2

som += mat[i][j]; --> C_3

$$T(N) = c_1 + c_2 (N+1) + c_2 N (N+1) + c_3 N^2$$

$$= O(N^2)$$

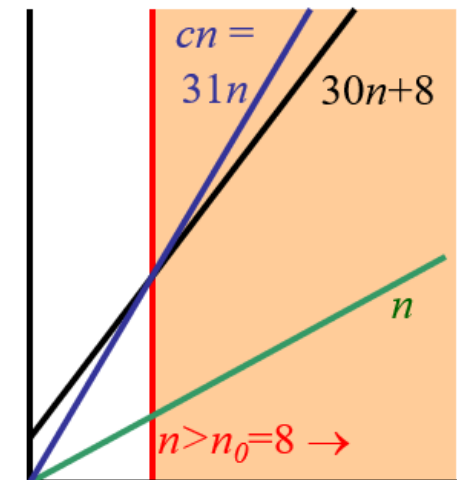
- Montrer que $30n+8 = O(n)$:**

Càd $\exists c, n_0 : 30n+8 \leq cn, \forall n > n_0$.

Soit $c=31, n_0=8$.

Pour $n > n_0=8$.

Alors $cn = 31n = 30n + n > 30n+8$, donc $30n+8 < cn$.



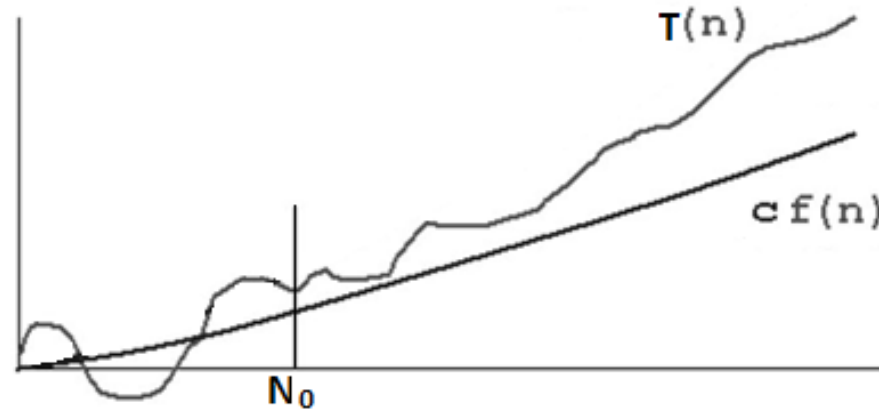
3. Notation Grand-Oméga

Soit $T(n)$ une fonction non négative. On dit que $T(N)$ est un $\Omega(f(N))$ s'il existe deux constantes positives C et N_0 telle que :

$$T(N) \geq C f(N) \text{ pour tout } N > N_0.$$

On dit que $T(N)$ **domine asymptotiquement** $f(N)$: $(T(N) \geq f(N))$

Interprétation géométrique



Exemples

$$T(N) = c_1 N^2 + c_2 N.$$

$$c_1 N^2 + c_2 N \geq c_1 N^2 \text{ pour tout } N \geq 0.$$

$$T(N) \geq c N^2 \text{ pour } c = c_1 \text{ et } N_0 = 1. \text{ Ainsi, } T(N) = \Omega(N^2)$$

4. Notation thêta

Lorsque le grand-O et le grand-oméga d'une fonction coïncident, on utilise alors la notation **grand-thêta**. Le temps d'exécution d'un algorithme est dans $\Theta(f(N))$ s'il est à la fois dans $O(f(N))$ et dans $\Omega(f(N))$.

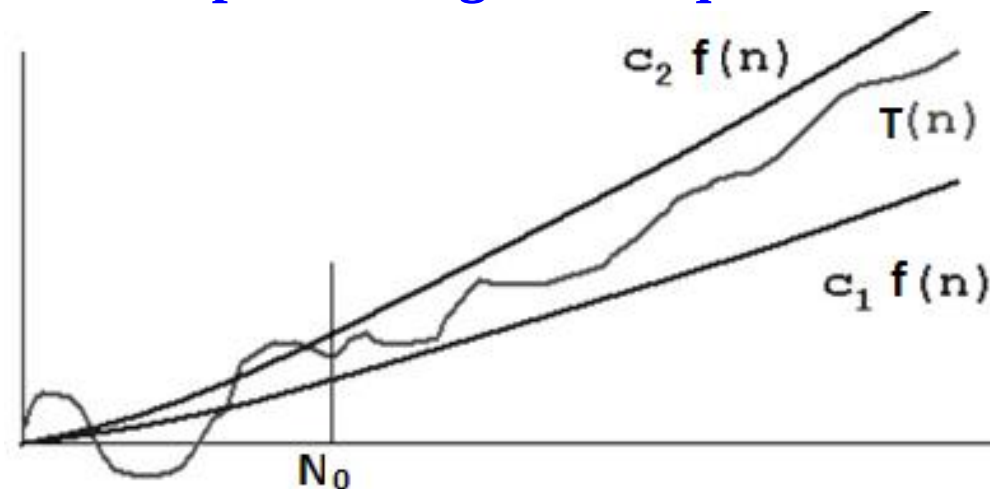
Autrement :

si $T(n)$ est une fonction non négative. On dit que $T(N)$ est un $\Theta(f(N))$ s'il existe trois constantes positives C_1 , C_2 et N_0 telle que :

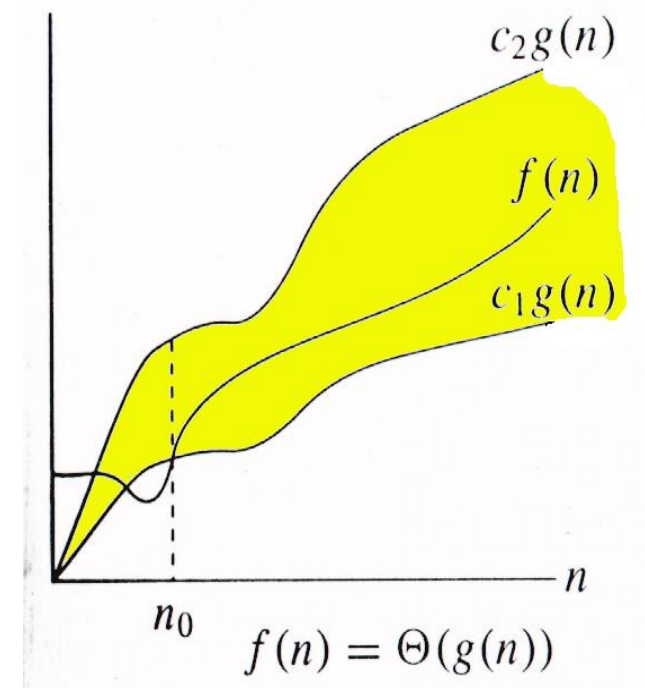
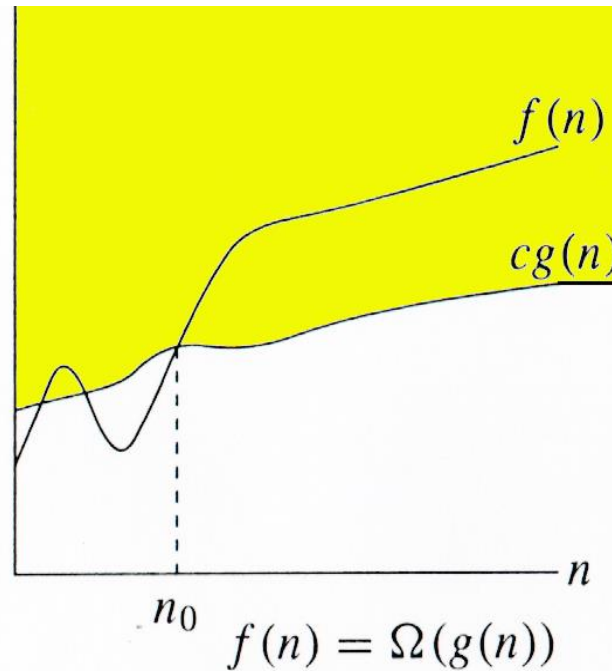
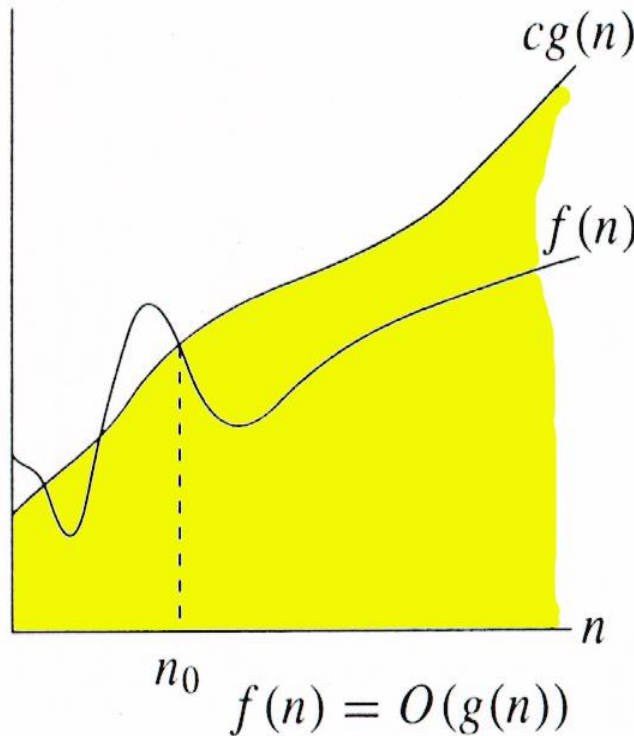
$$C_1 f(N) \leq T(N) \leq C_2 f(N) \quad \text{pour tout } N > N_0.$$

On dit que T et f sont **en même ordre** : $(T(N)) = (f(N))$

Interprétation géométrique



En résumé :



Relations entre O , Ω et Θ

Si $g(n)$ et $f(n)$ deux fonctions , $g(n) = \Theta(f(n))$ ssi
 $g(n) = O(f(n))$ et $f(n) = \Omega(g(n))$.

5. Propriétés

- **Transitivité**

$$f(n) = \Theta(g(n)) \ \& \ g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$$

$$f(n) = O(g(n)) \ \& \ g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$$

$$f(n) = \Omega(g(n)) \ \& \ g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n))$$

- **Réflexivité**

$$f(n) = \Theta(f(n))$$

$$f(n) = O(f(n))$$

$$f(n) = \Omega(f(n))$$

- **Symétrie**

$$f(n) = \Theta(g(n)) \text{ ssi } g(n) = \Theta(f(n))$$

- **Complémentarité**

$$f(n) = O(g(n)) \text{ ssi } g(n) = \Omega(f(n))$$

- **Autres règles**

si $f(n) = O(k g(n))$, où $k > 0 \Rightarrow f(n) = O(g(n))$

si $f_1(n) = O(g_1(n))$ & $f_2(n) = O(g_2(n)) \Rightarrow (f_1 + f_2)(n) = O(\max(g_1(n), g_2(n)))$

si $f_1(n) = O(g_1(n))$ & $f_2(n) = O(g_2(n)) \Rightarrow f_1(n)f_2(n) = O(g_1(n)) O(g_2(n))$

6. Efficacité des algorithmes

- Un algorithme est dit **efficace** si sa complexité (temporelle) asymptotique est **dans $O(P(n))$** où **$P(n)$ est un polynôme** et n la taille des données du problème considéré.
- On dit qu'un algorithme A est meilleur qu'un algorithme B si et seulement si :

$$T_A(n) = O(T_B(n)); \text{ et}$$

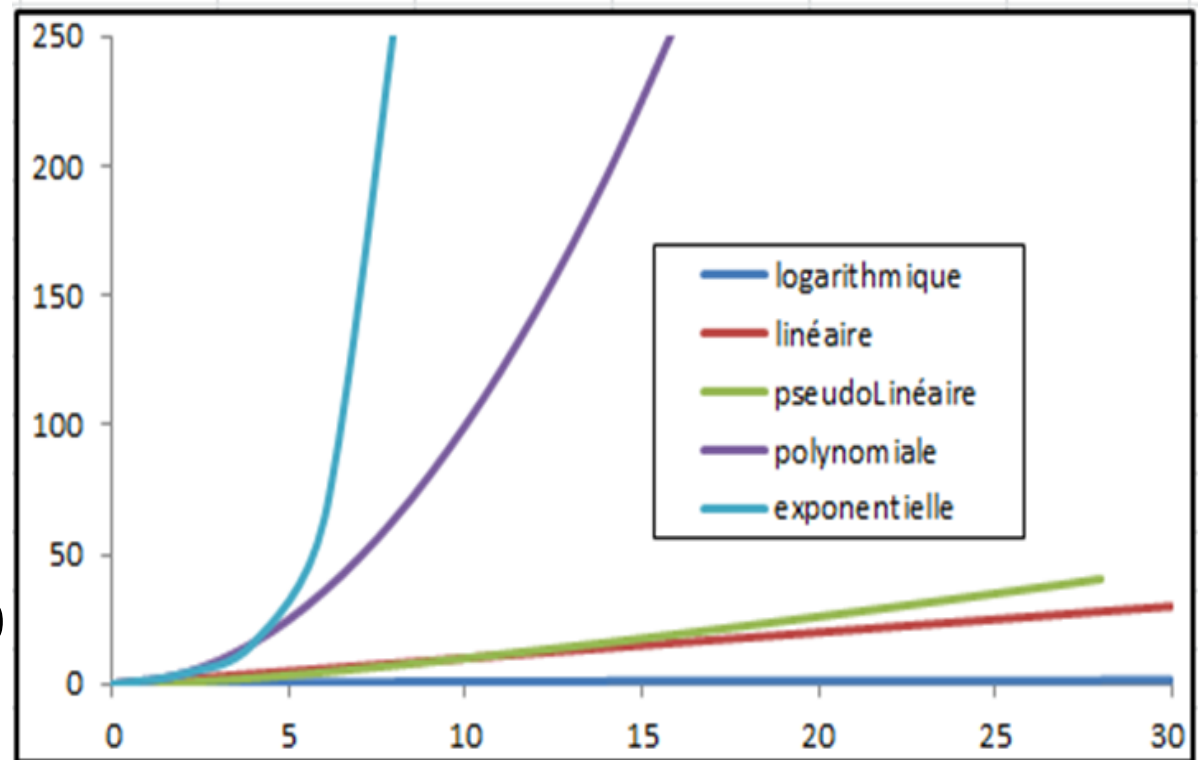
$$T_B(n) \neq O(T_A(n))$$

Où $T_A(n)$ et $T_B(n)$ sont les complexités des algorithmes A et B, respectivement.

7. Classes de complexité

Complexité	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 s	< 1 s	< 1 s	< 1 s	< 1 s	< 1 s	4 s
$n = 30$	< 1 s	< 1 s	< 1 s	< 1 s	< 1 s	18 min	10^{25} ans
$n = 50$	< 1 s	< 1 s	< 1 s	< 1 s	11 min	36 ans	∞
$n = 100$	< 1 s	< 1 s	< 1 s	1s	12,9 ans	10^{17} ans	∞
$n = 1000$	< 1 s	< 1 s	1s	18 min	∞	∞	∞
$n = 10000$	< 1 s	< 1 s	2 min	12 jours	∞	∞	∞
$n = 100000$	< 1 s	2 s	3 heures	32 ans	∞	∞	∞
$n = 1000000$	1s	20s	12 jours	31,710 ans	∞	∞	∞

- **Logarithmique** : $T_1(n) = O(\log_2 n)$
- **Linéaire** : $T_2(n) = O(n)$
- **Quasi-linéaire** : $T_3(n) = O(n \log_2 n)$
- **Quadratique** : $T_4(n) = O(n^2)$
- **Cubique** : $T_5(n) = O(n^3)$
- **Quasi-polynomial** : $T_6(n) = O(n^{\log_2 n})$
- **Exponentiel** : $T_6(n) = O(2^n)$



8. Résolution des équations de récurrence – théorème Master

Le théorème Master permet de donner la complexité d'algorithmes utilisant le paradigme “**diviser pour régner**”. Cependant, ce théorème ne permet pas de donner la complexité de tous les algorithmes de “diviser pour régner”.

Théorème : Soit $T : \mathbb{N} \rightarrow \mathbb{R}^+$ croissante à partir d'un certain rang. On suppose qu'il existe un rang $n_0 \geq 1$, des entiers $b \geq 2$ et $k \geq 0$ ainsi que des réels $a, c, d > 0$ tels que la relation de récurrence suivante soit vérifiée :

$$T(n) = \begin{cases} T(n_0) = d \\ T(n) = a T\left(\frac{n}{b}\right) + c n^k \text{ pour tout entier de la forme } n = n_0 b^p, p \in \mathbb{N} \end{cases}$$

Alors on a la disjonction de cas suivante :

- (i) Si $a < b^k$ alors $T(n) = \Theta(n^k)$
- (ii) Si $a = b^k$ alors $T(n) = \Theta(n^k \log_b(n))$
- (iii) Si $a > b^k$ alors $T(n) = \Theta(n \log_b(a))$

Remarque : Si on a qu'une majoration dans l'hypothèse de récurrence (i.e. $T(n) \leq a T\left(\frac{n}{b}\right) + c n^k$), on remplace Θ par O .

Exercices (Suite)

Exercice 3 : Analyse d'itérations emboîtées

Evaluer la complexité pour chacun des codes suivants :

1. Code 1

```
for(i = 1; i <= n; i++)
    for( j = 1 ; j <= n ; j++) x += a;
```

n^2

2. Code 2

```
for(i = 1; i <= n; i++)
    for( j = 1 ; j <= i ; j++) x += a;
```

$\frac{n(n+1)}{2}$

3. Code 3

```
for(i = 1; i <= n; i*=2)
    for( j = 1 ; j <= i ; j++) x += a;
```

$1 + 2 + 4 + \dots + n$

4. Code 4

```
for(i = 1; i <= n; i++)
    for( j = 1 ; j <= i ; j++)
        for( k = j ; k >= 1 ; k--) x += a;
```

$\frac{n(n+1)(n+2)}{6}$



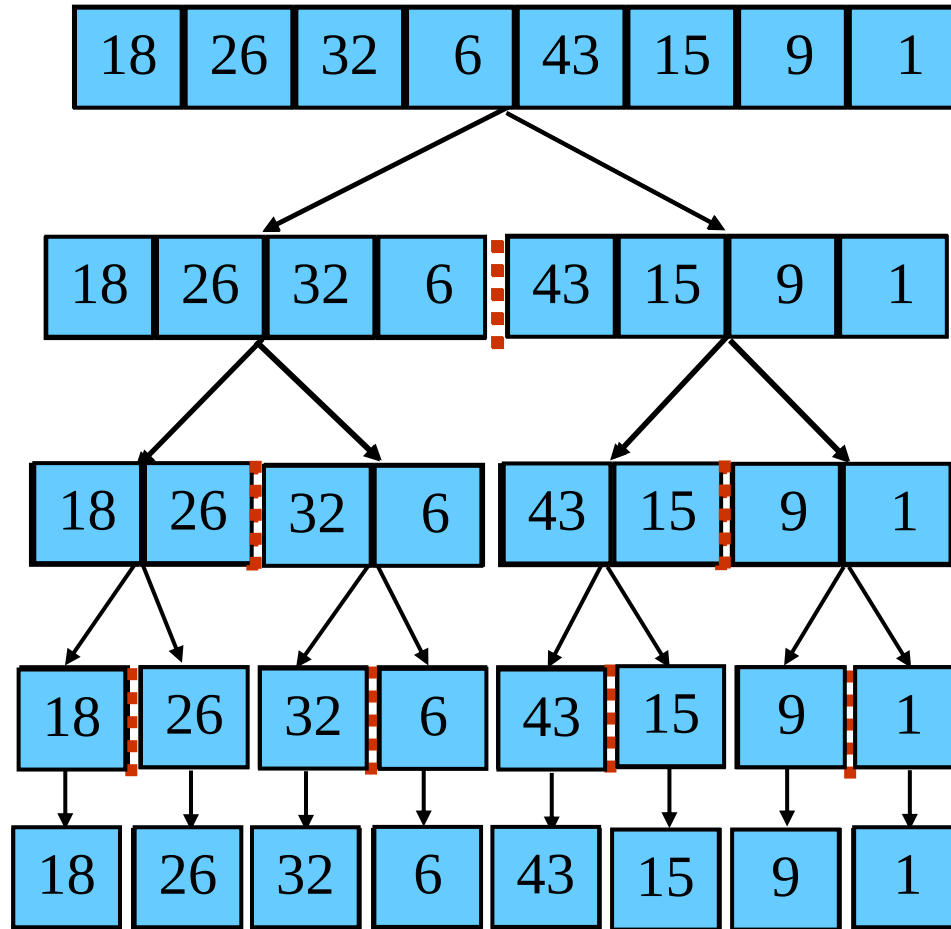
Exercice 4 : Tri par fusion

L'algorithme de tri par fusion fait partie des algorithmes basés sur le principe « Diviser pour régner ». Il consiste à :

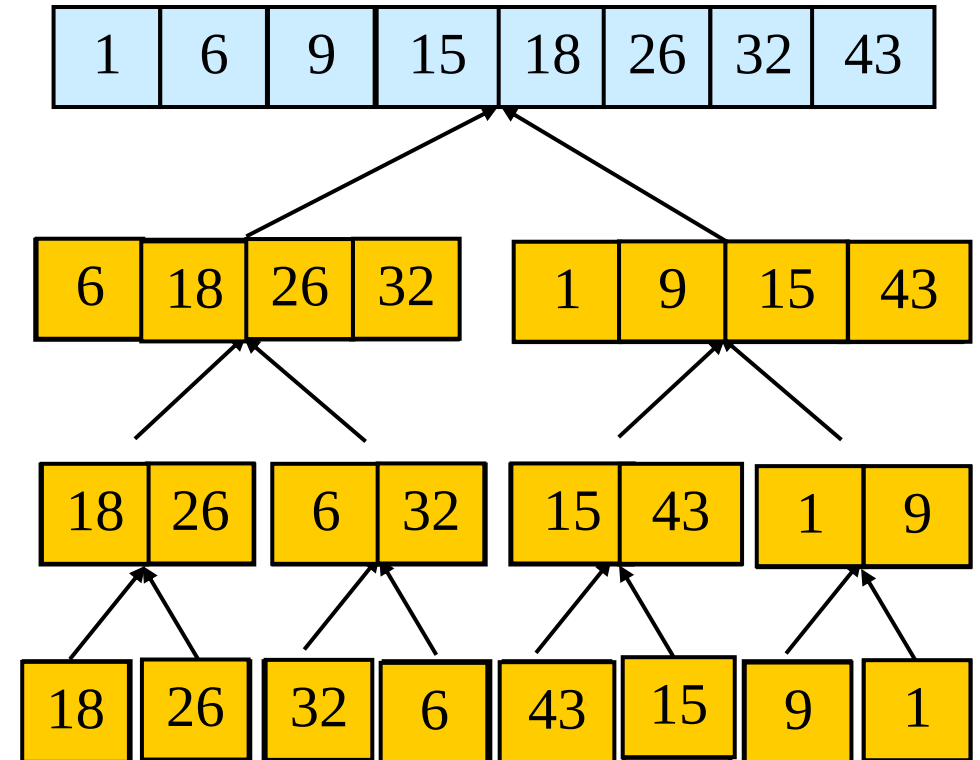
- diviser la tableau à trier en deux moitiés
 - Trier chacune des deux moitiés selon le même principe tant que sa taille est supérieure à 1
 - Fusionner les deux moitiés triées dans un tableau trié.
1. Ecrire une fonction qui divise en tableau en deux moitiés
 2. Ecrire une fonction qui fusionne deux tableau triés en un tableau trié
 3. Ecrire une fonction tri par fusion
 4. Evaluer les complexités représentatives des trois fonctions.
 5. Simplifier chaque complexité $T(n)$ en appliquant l'analyse asymptotique via les notations O , Ω et θ

Tri fusion– Exemple

Sequence Originale



Sequence triée



Exercice 4 : Tri par fusion – Correction (on suppose que $n=2^p$)

```
#include<stdio.h>
```

```
void diviser(const int *t,int *t1,int *t2,int n){
```

```
    int j=0,i=0,m1=n/2;
```

//=> c_1

```
    for(;i<m1;i++)t1[i]=t[i];
```

//=> $c_2*n/2+c_3$

```
    for(;i<n;j++,i++)t2[j]=t[i];
```

//=> $c_4*n/2+c_3$

```
}
```

//=> $c_5*n+2*c_3+c_1$

```
void fusionner(int *t,const int *t1,const int *t2,int n){
```

```
    int i=0,j=0,k=0,m1=n/2,m2=n-m1;
```

//=> c_6

```
    while(i<m1 && j<m2)t[k++]=t1[i]<t2[j]?t1[i++]:t2[j++];
```

//=> $c_7*(n-x)+c_8$

```
    for(;i<m1;i++,k++)t[k]=t1[i];
```

//=> $c_7*x+c_8 \mid 0$

```
    for(;j<m2;j++,k++)t[k]=t2[j];
```

//=> $0 \mid c_7*x+c_8$

```
}
```

//=> $c_7*n+2*c_8+c_6$

```
void trifusion(const int *t,int n){
```

```
    if(n>1){
```

//=> c_9

```
        int m1=n/2,m2=n-m1,*t1=new int[m1],*t2=new int[m2];
```

//=> c_{10}

```
        diviser(t,t1,t2,n);
```

//=> $c_5*n+2*c_3+c_1$

```
        trifusion(t1,m1);trifusion(t2,m2);
```

//=> $2*T(n/2)$

```
        fusionner(t,t1,t2,n);
```

//=> $c_7*n+2*c_8+c_6$

```
        delete[] t1; delete[] t2;
```

//=> c_{11}

```
}
```

//=> $T(n)=2*T(n/2)+(c_5+c_7)*n+(c_{11}$

```
}
```

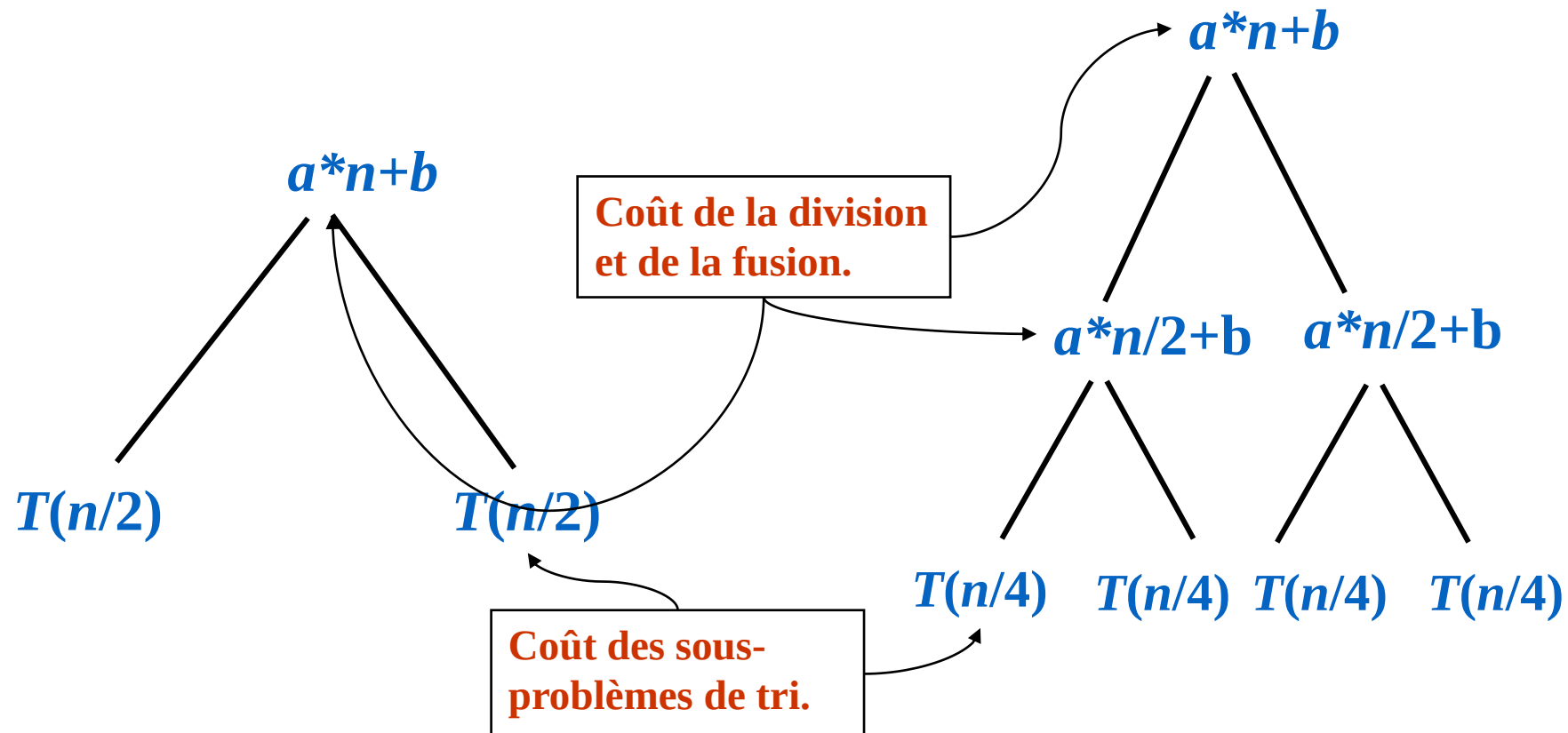
On note : $a=c_7+c_5$ et $b= c_{11}+c_{10}+c_9+2*c_8+c_6+2*c_3+c_1$ $+c_{10}+c_9+2*c_8+c_6+2*c_3+c_1)$

Exercice 4 : Complexité du Tri par fusion

Pour le problème d'origine, nous avons un coût de $a*n+b$, plus deux sous-problèmes chacun de taille $n/2$ et de durée $T(n/2)$.

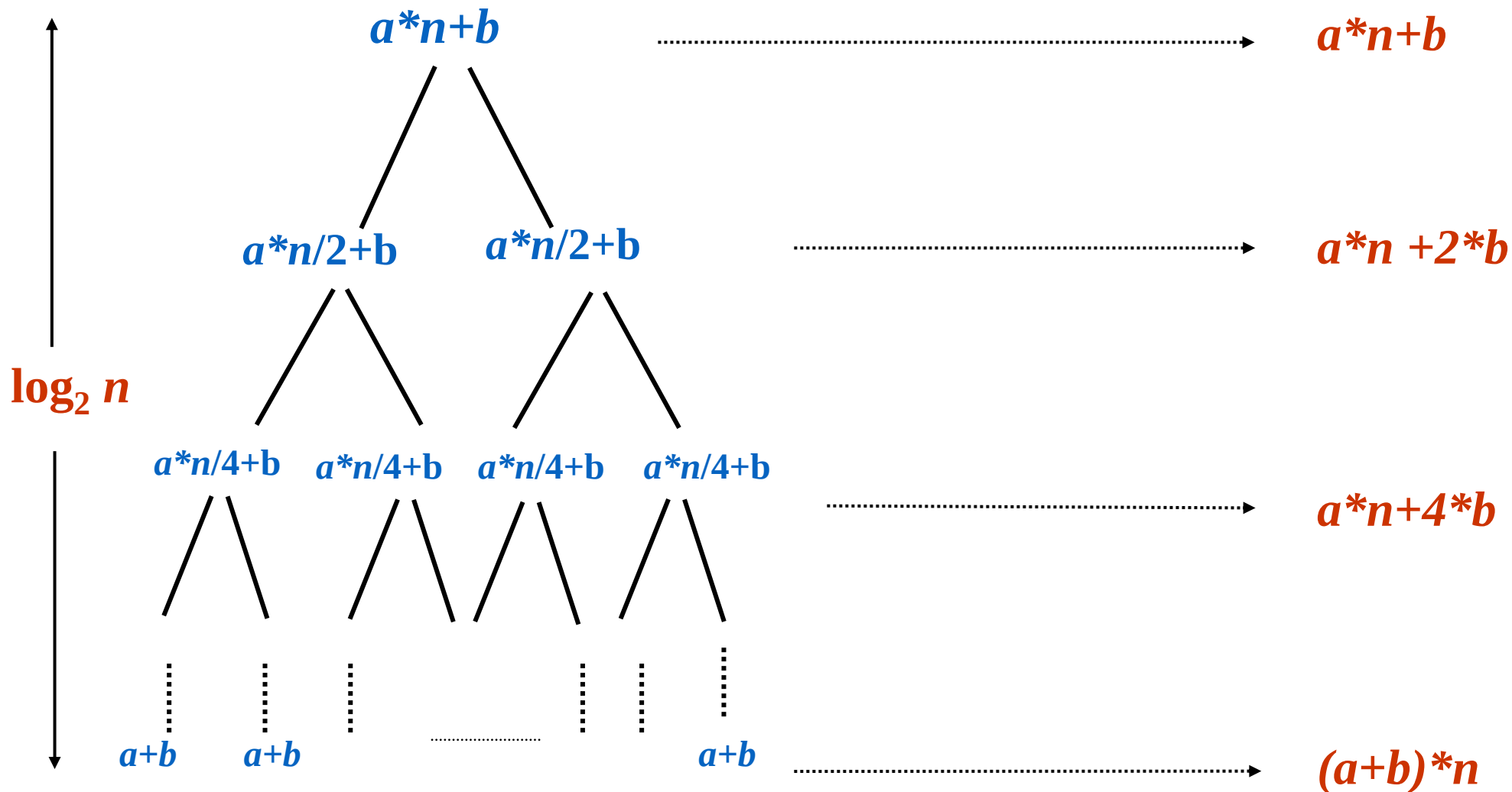
Chacun des problèmes de taille $n/2$ a un coût de $a*n/2+2*b$ plus deux sous-problèmes, chacun coûtant $T(n/4)$.

...



Chap I : Analyse de la complexité des algorithmes

On continue à développer jusqu'à ce que la taille du problème soit réduite à 1.



Total :

$$\begin{aligned} T(n) &= a * n * \log_2 n + b \sum_0^{\log_2 n} 2^i \\ &= a * n \log_2 n + b(2^{\log_2 n + 1} - 1) = a * n * \log_2 n + b * (2 * n - 1) \\ &= \theta(n * \log_2 n) \end{aligned}$$

Exercice 5 : Recherche du maximum dans un tableau

On considère un tableau T d'entiers de taille n

1. Écrire une fonction itérative qui recherche et retourne l'entier maximal de T.
2. Donner une forme récursive de cette fonction
3. Donner la complexité théorique des deux fonctions en prenant comme opération fondamentale les opérations de comparaisons
4. Quelle est la classe de complexité de ces deux fonctions
5. Donner une approximation du temps de recherche du maximum que fera un ordinateur équipé d'un processeur 2GHz pour trouver le résultat dans un méga tableau de 2^{63} entiers. On supposera que le processeur est utilisé à son entière capacité pour ce traitement. On fera pour simplifier les calculs l'approximation suivante : $2^{10} \approx 10^3$
6. Refaire les questions de 1 à 5 pour le cas où T est pré-trié.

Exercice 6 : Recherche dans un tableau (Séquentielle)

On étudie un algorithme de recherche séquentielle dans un tableau T d'entiers de taille n . On se place dans le cas où il n'y a pas d'hypothèse sur le fait que le tableau est ordonné ni sur la présence de l'élément cherché dans le tableau.

1. Spécifier et écrire un tel algorithme,
2. Donner une forme récursive de cette fonction
3. Déterminer les cas favorables et défavorables et les complexités pire et meilleur en nombres de comparaisons correspondants.
4. On suppose que le cas défavorable est probable à 30% et que tous les autres cas restant sont équiprobables, donner une complexité moyenne.

Exercice 7 : Recherche dans un tableau trié (binaire)

On étudie un algorithme de recherche binaire dans un tableau T ordonné d'entiers de taille n . On se place dans le cas où il n'y a pas d'hypothèse sur la présence de l'élément cherché dans le tableau.

1. Spécifier et écrire un tel algorithme,
2. Donner une forme récursive de cette fonction
3. Déterminer les cas favorables et défavorables et les complexités pire et meilleur en nombres de comparaisons correspondants.
4. On suppose que le cas défavorable est probable à 20% et que tous les autres cas restant sont équiprobables, donner une complexité moyenne.

Exercice 8 : Supprimer un élément d'une liste

On considère une liste simplement chaînée d'entiers

1. Écrire des fonctions qui suppriment :
 - La 1^{ère} occurrence de l'élément donné en paramètre dans la liste ;
 - Toutes les occurrences de l'élément donné en paramètre dans la liste.

A chaque fois on renverra la liste initiale si l'élément à supprimer n'apparaît pas dans la liste.

2. Evaluer la complexité représentative pour les deux solutions

Exercice 9 : Afficher une liste simplement chaînée dans l'ordre inverse

1. Écrire une fonction qui affiche dans l'ordre inverse les éléments entiers d'une liste simplement chaînée.
2. Evaluer sa complexité.